

EEA112 邏輯電路設計 期末專題報告書

專題名稱：循序邏輯電路自動化設計系統 (Sequential Circuit Design Automation System)

開發者：資訊工程學系及電機工程學系雙主修 陳伯豪 (學號：1131525)

專案版本：v2.4.0 (Enterprise Academic Build)

開源庫連結：<https://github.com/Moyu0730/YZU-1142-Digital-Circuit-Design-Final-Project>

網站連結：<https://moyu0730.github.io/YZU-1142-Digital-Circuit-Design-Final-Project>

一、系統功能與核心特色 (System Features & Core Characteristics)

本系統是一套專為數位電路設計與合成打造的整合性 EDA (Electronic Design Automation) 線上工具平台，其核心設計突破了傳統學習或工程設計中手工推導的效率瓶頸，實現了從「狀態規格輸入」到「電路圖產出」的全自動化編譯流程。整體功能與技術特色詳述如下：

(一) 全自動化邏輯化簡與多型態 Flip-Flop 合成

- **Dynamic State Table Compilation**

系統提供高度可擴充的交談式矩陣表格，使用者可依據設計需求動態增刪狀態列。

- **Multi-Type Flip-Flop Excitation Dictionary**

系統內部全面建置並封裝了 JK Flip-Flop、D Flip-Flop 與 T Flip-Flop 的 Characteristic Equations 與 Excitation Tables。當使用者輸入 Current State、Next State 與 Outputs 後，系統會即時、動態地查表並填入 Flip-Flop Excitation 輸入。

- **Quine-McCluskey (QM) Higher-order Minimization Algorithm**

核心化簡捨棄了變數受限的圖解卡諾圖，採用圖表化簡法。透過二進制 Minterms 的遞迴分群、位元比對與合併消除，並結合 Petrick 演算法處理 Coverage Chart，能完美處置包含 Don't Care Terms 在內的極端 Truth Table，精準輸出成本最低的 Sum of Products (SOP) Boolean Equation。

- **Karnaugh Map Synthesis and Matrix Visualization**

除了後端的演算法化簡，系統亦能將對應的布林函數動態渲染為二維卡諾圖矩陣，以視覺化的方式展示變數分組與圈選軌跡。

(二) 階層式向量電路圖 (SVG) 自動佈線與幾何配置

- **Hierarchical CAD Layout Netlist**

電路產生器在解析布林方程式後，會自動將元件分為四大垂直階層進行幾何對齊。

最左側第一層為 Primary Inputs 與 NOT Gates；第二層為負責運算乘積項的 AND Gates；第三層為負責運算和項的 OR Gates；最右側第四層為 Flip-Flops 暫存器單元與 Primary Outputs。

- **Orthogonal Routing Automatic Polyline Algorithm**

系統會動態計算元件間節點的幾何座標，利用該路徑演算法自動生成無重疊、轉折俐落的向量走線，並在實體線路連接處精準補上節點圓點。

(三) 高階互動幾何雷達與 SOP 語法樹反向解析

- **Infinite Scalar Scaling based on SVG ViewBox**

系統內建專業 CAD 等級的畫布視角控制，透過操作 SVG 的 viewBox 屬性實現 40% 至 400% 的向量無損縮放，並提供一鍵適應螢幕 (Fit to Screen) 功能，解決傳統 CSS transform 導致的畫布截斷與邊界溢出問題。

- **Geometric Radar Tooltips**

系統透過 `getBoundingClientRect()` 方程式即時監聽游標在畫布上的空間幾何位置，當使用者點擊任何 Logic Gate 或 Flip-Flop 時，資訊提示窗會以微秒級的延遲精準彈出。

- **布林乘積項垂直佈局配對**

本系統具備「看懂電路圖」的硬核特色。當點擊某一 AND Gate 時，反向解析引擎會追蹤其相連的輸出端口網表，利用字串語法樹將方程式切開，並根據該 AND Gate 在畫布上的實體垂直 Y 軸座標進行排序，精準抓出該閘目前正在運算的特定乘積項項，避免傳統 EDA 工具全數顯示或無法對齊的瑕疵。

- **Flip-Flop 實體物理特徵方程式渲染**

當點擊 Flip-Flop 元件時，輸出端不再顯示籠統的說明，而是直接帶入對應 Flip-Flop 的物理特徵方程式，並自動為相加的乘積項外圍套上高亮括號以釐清運算優先級。

（四）文字邊界框實體遮罩（Masking）與圖層重排

- **Text BBox Entity Mask with White Background**

在複雜電路自動佈線時，走線常會不可避免地穿越引腳標籤。系統內建遮罩引擎，會透過 `getBBox()` 方程式動態抓取每個文字的精確幾何寬高，並在文字正後方自動墊入圓角矩形，將穿越文字背後的導線進行實體「視覺切斷」，保持字體的極致可讀性。

- **Asynchronous DOM Layer Reflow Engine**

針對 SVG 不支援 CSS z-index 的物理限制，系統引入 MutationObserver 監聽畫布的突變。一旦電路圖生成或重繪，系統會強制將所有文字標籤與遮罩矩形群組移至整體 DOM 樹的最末端，在渲染順序上確保線路永遠沉在底部、文字與遮罩永遠浮在最上層，重現專業圖紙質感。

（五）零污染深層克隆報表匯出與 3x3 記憶主題矩陣

- **Deep Clean Export Clone**

當使用者在介面上放大畫布進行局部除錯時，DOM 樹會殘留龐大的物理內聯樣式。本系統的匯出引擎在觸發「匯出 PDF 報告」或「下載圖片」時，會於背景複製一份隱形的純淨 DOM 副本，使用 `removeProperty` 徹底洗刷所有因視角縮放產生的變形數據，並重新將寬度校正為 100%，確保產出的 A4 規格電路實驗報告書中的電路圖不破版或被裁切。

- **3x3 Custom Color Matrix**

Settings 面板內建 9 種經典工程與現代風格色系，同時，系統具備全域狀態追蹤器（Global State Tracker），在儲存並關閉後，再次打開設定時能完美記憶並維持當前的網格密度與色彩選擇。

二、系統架構與模組設計（System Architecture & Module Specification）

本系統遵循高內聚、低耦合的軟體工程規範，將 EDA 工具的演算法、繪圖、互動與報表功能拆分為五大核心模組：

（一）Global Initialization Coordinator (*app.js*)

作為主程式進入點，負責在網頁加載時協調各個模組。內建 `initSplittersAndDrag()` 機制，利用監聽 `mousedown` 與 `mousemove` 控制三欄式 Draggable Flex-box 的實體寬度，並在重排時安全呼叫適應螢幕功能。同時搭載 `bindSafe()` 預防機制，在綁定按鈕時先進行函數類型驗證，杜絕腳本載入順序不同所引起的 `ReferenceError`。

（二）Simplification Engine (*logicEngine.js*)

系統的硬體數學核心。負責接收前端表格的二進制輸入，根據 Excitation Table 將狀態資料擴充為 Flip-Flop Input。隨後執行 QM Algorithm：第一階段進行逐欄位元比對找出 Prime Implicant；第二階段透過 Petrick Function 處理 Coverage Matrix，剔除 Redundant 硬體，最終將 minSOP Boolean Equation 傳遞給下游。

（三）Vector Circuit Rendering Engine (*circuitRender.js*)

負責將上游輸出的布林字串拆解為 Logic Gate 物件，並動態創建節點。透過階層式位置模型，計算出每個 AND Gate、OR Gate 與 NOT Gate 在寬敞網格上的實體 XY 座標，並佈線電路。

（四）Circuit Interaction and Visual Optimization Engine (*circuitInteraction.js*)

畫布的核心內建三大核心子系統：

1. **Vector Zoom Operator**

接收放大、縮小或 Reset 指令，強制覆寫 SVG 幾何樣式並動態調校 viewBox。

2. **BBox Mask and Layer Sequencer**

利用 MutationObserver 在畫布變更時異步執行 *applyTextBackgroundMasks()*，運算文字 Bounding Box，墊入白底矩形並強推送至頂層。

3. **SOP Syntax Tree Parser**

在點擊 Logic Gate 時攔截事件，將方程式字串切開，比對元件垂直 Y 座標，反向提取對應的布林項並執行黃金配色高亮渲染。

（五）Interface Preferences and Pop-up Window Manager (*modalManager.js*)

負責主控台的所有偏好設定與資訊顯示。具備 Lazy Initialization，點擊按鈕時若 Modal 樹尚未建立則即時強制注入，防範 DOMContentLoaded 順序落差。About 面板採用高級的白底置中 Dashboard 雙欄規格卡片流設計，將「開發者個人品牌」與「彩色規格條列的系統技術架構」精美呈現。

（六）Report and Stream Export Manager (*exportManager.js*)

處理輸出端檔案流。在下載 SVG 或 PNG 時會建立乾淨的 DOM 複本以消滅 UI 縮放偽影；在點擊匯出報告書時，會抽取 Truth Table、邏輯化簡公式與克隆電路，重寫 PDF 容器視圖，喚醒瀏覽器的無頭列印機制進行無損 A4 輸出。

三、完整製作與開發流程 (Complete Development Process)

Phase 1: Requirements Analysis, Excitation Table, and Implementation of Logic Minimization Algorithms

1. **Specification Development**

確立系統必須支援最常見的 Synchronous FSM Model，且必須涵蓋 JK, D, T 三種 Flip-Flop。

2. **Mathematical Model Encapsulation**

在 *logicEngine.js* 中將 Flip-Flop 的 Excitation 邏輯寫死為 Lookup Maps。

3. **Implementing the QM Algorithm in Code**

實作 Prime Implicant 生成器。利用 JavaScript 的位元操作與陣列組合，撰寫能將 0、1 與 - 進行遞迴比對的函數。建構二維 Coverage Matrix，撰寫尋找 Essential Prime Implicants 的篩選器，確保化簡結果的絕對正確性。

Phase 2: Netlist Partitioning, Hierarchical Geometric Placement, and Automatic Routing R&D

1. **Netlist Parser Development**

撰寫 String Regular Expressions，將 QM 引擎輸出的最簡方程式（例如： $J_0 = Q'_1 \cdot X$ ）切割為多個 Product terms and sum terms。

2. **Stratification Layout Model**

在虛擬畫布上劃分四個主要的垂直通道，訂定各個通道的 X-offset 與元件間的 Y-spacing，確保電路圖在擴充時擁有均勻的呼吸留白空間。

3. **Routing Path Calculation**

編寫自動走線邏輯，當線路需要跨越通道連接到後端的 Logic Gate 時，自動計算中間折點座標，生成帶有標準 90 度直角轉折的主要幹線與引腳支線。

Phase 3: Geometric Detection, AST Inverse Equation Analysis, and Advanced Interactive Implementation

1. Canvas Scaling Debugging

解決早期版本使用 CSS transform: *scale()* 導致整體 Flexbox 排版破裂與文字變糊，全面改寫為直接調校 SVG 實體寬高屬性與 viewBox 純量縮放的架構。

2. Practical Implementation of Spatial Geometry Radar

綁定 svgContainer 的 click 事件，透過計算滑鼠落點與畫布上各個矩形、多邊形幾何中心的歐幾里得距離，精準捕捉使用者點擊的實體元件類型。

3. R&D on Reverse-engineered Formula Matching

為了解決「點擊 AND Gate 卻顯示整條方程式」的工程瑕疵，實作了基於實體垂直座標比對的 AST 反向配對。透過 *sort()* 將畫布上的 AND Gate 按 *getBoundingClientRect().top* 排序，與切開後的布林乘積項陣列進行 Index 一對一精準配對。

Phase 4: Visual Mask Rendering Optimization, DOM Monitoring, and Layout-break Prevention Engineering

1. Debugging of Trace Routing through Text

傳統自動佈線中，深藍與綠色的走線直接從 Q_0, Q_1 等腳位文字中心穿過，導致視覺雜亂。

2. Solid White-background Mask Injection

在 *circuitInteraction.js* 中利用 MutationObserver 建立自動遮罩管線。當電路渲染完畢，立刻對所有 `<text>` 節點呼叫 *getBBox()*，獲取其精確的 Bounding Box，並在其正前方動態插入一個填滿白色的圓角矩形物件。

3. Implementation of DOM Z-index Reordering

為克服 SVG 的渲染層級缺陷，撰寫重排代碼。在遮罩矩形插入後，利用 *parentNode.appendChild(txt)* 迴圈，將文字與遮罩元件強行拔除並重新插回 DOM 樹的最頂層，成功在物理層面上讓走線從文字下方穿過。

Phase 5: Deep Clone Export Optimization, UI Refactoring, and System Deployment

1. Exporting Layout: Debugging Contamination Issues

修正當使用者放大畫布後，點擊匯出 PDF 報告會導致 A4 圖紙只截取到局部放大圖的嚴重缺陷。在 *exportManager.js* 中導入 *cloneNode(true)* 深層複製技術，清除所有縮放產生的內聯樣式屬性，完美實現自適應 A4 報表輸出。

2. UI/UX Streamlining and Refactoring

移除 Settings 面板中導致網格不對稱的琥珀金，重構為純淨的 3x3 矩陣，並補上變數狀態記憶鎖。將 About 視窗徹底翻修，拆除破版的負邊距，改為現代化 Dashboard 卡片佈局，完美融合 9 種系統主題。

3. System Packaging and Deployment

全面校對程式碼，確保所有註解、變數命名與 Console 日誌 100% 符合國際開源標準的全英文規範，並發布至 GitHub Pages。

四、核心技術挑戰與解決方案 (Technical Challenges & Solutions)

挑戰一：Logic Gate 提示窗布林表達式擷取錯誤與混淆

- 現象：**當系統合成輸出函數（例如： $Z = Q_0 \cdot X' + Q_1$ ）時，電路圖中會包含多個 AND Gates 與 OR Gates。初期系統在點擊任何一個 AND Gate 時，Inputs 欄位都會錯誤地把 Q_0, X', Q_1 全部擠在一起。這在數位邏輯理論上是完全錯誤的，因為單個 OR Gate 不應負責處理 OR 的加法項。
- 解決方案：**實作 SOP 幾何座標空間配對演算法，當點擊事件發生時
 - 引擎首先提取該 Logic Gate 在 UI 表格中對應的右側輸出口口的完整方程式字串。
 - 使用字串解析器以 + 號為基準進行切分，將方程式解構為獨立的乘積項陣列。
 - 透過 *querySelectorAll* 搜集畫布上所有連向 Z 端口的 AND Gate，利用 *getBoundingClientRect().top* 獲取它們在瀏覽器視窗中的實體垂直高度並進行由上至下的升冪排序。
 - 利用當前被點擊 AND Gate 在排序陣列中的 Index，精準對齊乘積項陣列中對應的位置。
- 成果：**成功讓第一個 AND Gate 精準只擷取 Q_0, X' 作為輸入，並在 Output 欄位輸出高亮的 $Q_0 \cdot X'$ 乘積項；同時優化 OR Gate，自動為相加項套上括號 $()$ 。

挑戰二：SVG 物理渲染層級限制導致電路線穿字壓字

- **現象：**由於系統導線採用自動走線配置，垂直幹線與水平引腳線在交錯時會直接穿過 Flip-Flop 輸出引腳的文字標籤。因為 SVG 規範中不支援 CSS 的 `z-index` 屬性，其渲染完全依賴於節點在檔案中的先後順序。電路產生器是先繪製文字才繪製線路，導致高飽和度的藍色與綠色導線無情地直接壓在文字上方，造成嚴重的圖紙破相與文字失真。
- **解決方案：**利用 `MutationObserver` 捕捉畫布動態，即時運算每個標籤的 `Bounding Box`，在文字正後方注入實心白色的圓角遮罩矩形，並透過 JS 將文字與遮罩元件抽離並重新 `append` 到群組的最末端。
- **成果：**導線在穿越文字周圍時會被完美地「視覺切斷」，且文字保持原汁原味的高解析度清晰度，達到國際頂尖 EDA 工具的繪圖標準。

挑戰三：畫布縮放狀態污染導致匯出 PDF 報表嚴重破版被裁切

- **現象：**為了提供良好的操作體驗，系統提供了向量縮放按鈕。當使用者在介面上將電路圖放大時，縮放引擎會在 `#circuit-svg` 節點上強制注入絕對的內聯樣式。此時如果使用者點擊「預覽 PDF 報告」，匯出引擎若直接抓取當前的 SVG，該巨大的尺寸數據會直接污染 PDF 的頁面佈局，導致電路圖衝破 A4 紙張邊界，產生嚴重的畫面截斷與排版偏移。
- **解決方案：**在 `exportManager.js` 中建立 **Clean Clone Layer**
 1. 阻斷對原始活躍 DOM 節點的直接提取，改用 `s.cloneNode(true)` 進行整棵 SVG 樹的深層複製。
 2. 使用 `style.removeProperty()` 徹底拔除複本上的 `width`, `height`, `max-width`, `max-height` 與 `transform` 等所有受到滑鼠縮放污染的內聯屬性。
 3. 強制為複本注入 `svgClone.setAttribute('width', '100%')` 與 `svgClone.setAttribute('height', 'auto')`。
- **成果：**被複製的淨化電路圖在塞入 PDF 預覽視窗與觸發無頭列印時，能夠 Responsive 地縮放至 A4 容器內，產出排版無瑕疵、符合業界標準的學術報告書。

五、系統測試與成果驗證（Testing & Validation）

為確保邏輯化簡演算法的精確度以及前端渲染的穩定性，本系統進行了多層級的嚴格測試：

1. **布林化簡正確性驗證：**使用傳統數位電路課本中的經典有限狀態機題目輸入 Truth Table，比對系統化簡輸出的 SOP 方程式，結果與人工卡諾圖推導完全一致，且自動化簡速度達微秒級。
2. **Don't Care 收斂測試：**輸入包含超過 50% 任意項的未定義狀態轉換表，驗證 QM Algorithm 在 Prime Implicant Chart 篩選時是否能正確利用 Don't Care 進行最大化群組圈選，測試結果顯示系統能成功產出最精簡的硬體閘數。
3. **互動提示窗空間對齊測試：**在 40% 的極小視角與 400% 的極大視角下，點擊畫布邊緣的 OR Gate，系統幾何雷達皆能精準彈出 Tooltip，且內部乘積項垂直配對無任何 Index Shifting。

六、結論與未來展望（Conclusion & Future Work）

（一）專題結論（Conclusion）

本專題成功開發出一套具備工業級 UI/UX 的「循序邏輯電路自動化設計系統」。系統不僅完美融合了資訊工程的進階資料結構與演算法，更具體實踐了電機工程的數位電路設計核心理論。

透過解決提示窗變數精準擷取、SVG 文字遮罩重排以及 PDF 深層克隆等三大技術難題，本系統的完整度與渲染品質已達到極高的實務水準，是一款兼具學術嚴謹度與操作實用性的優秀 EDA 教學輔助與設計工具。